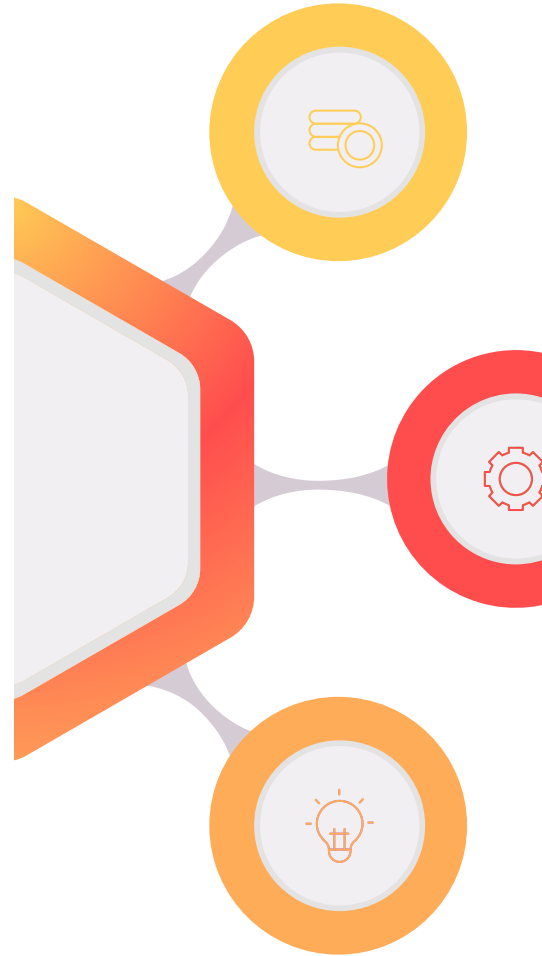
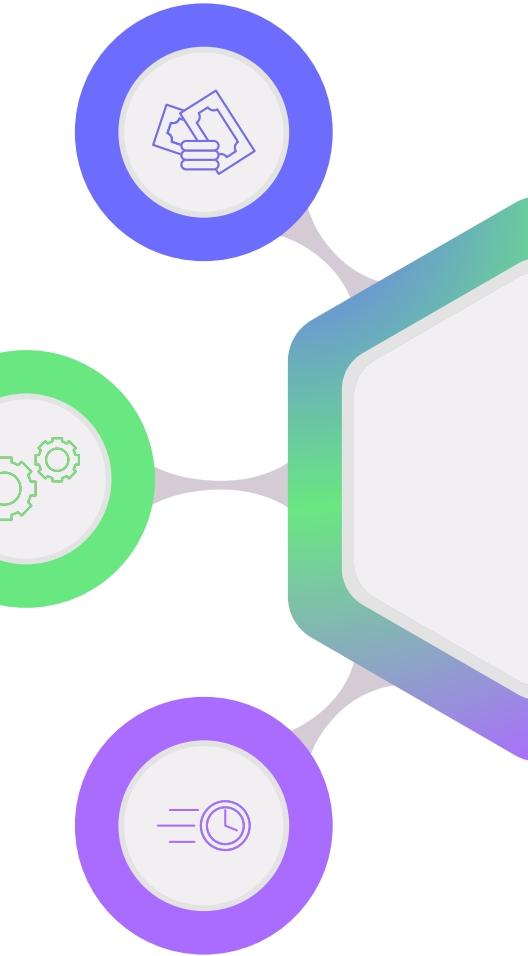


Test Driven Development (TDD)

Oscar Eduardo Alvarez Rodriguez
Ingeniería de software 1 - 2024 - 2



Índice

01

Introducción



02

Motivación



03

Conceptos básicos



04

Demostración práctica



05

Beneficios y retos



06

Recomendaciones



Introducción

¿Qué es?

Forma de desarrollar



¿Pasos?

Red → Green → Refactor



¿Qué busca?

Reducción de bugs desde el inicio

Código más mantenible y mejor
diseñado

Confianza para refactorizar sin miedo



¿Vale la pena hacer test?

Cuanto más tarde detectas un error,
más caro es arreglarlo.



Los test en general...

NASA Mars Climate Orbiter (pérdida
por error de unidades métricas vs
imperiales).



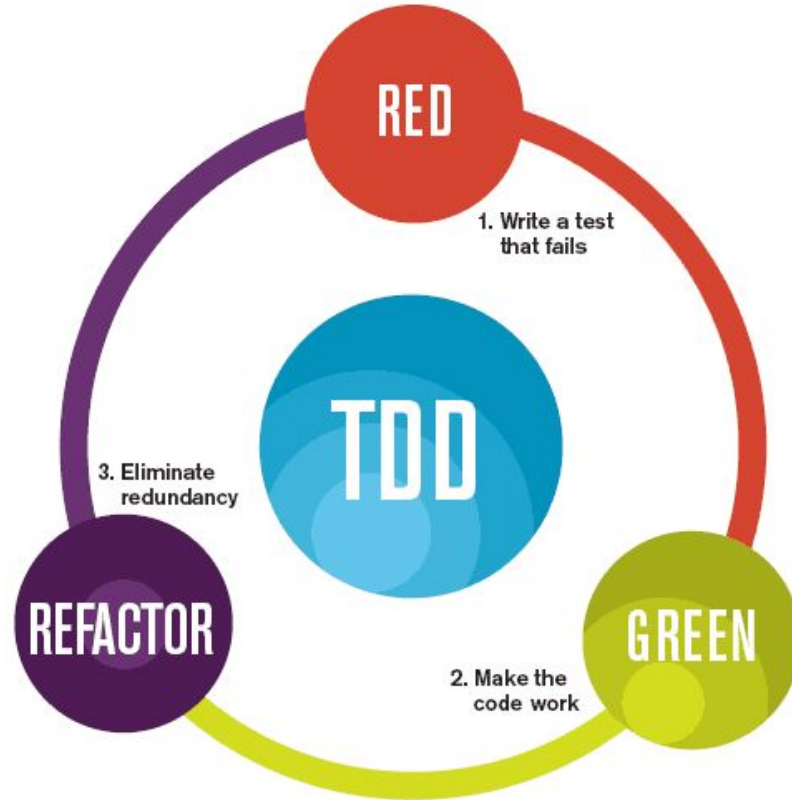
1

2

3

4

PASOS DEL TDD



The mantra of Test-Driven Development (TDD) is "red, green, refactor."

TOMADO DE:

<https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.nimblework.com%2Fes%2Fagile%2Fdesarrollo-dirigido-por-pruebas-tdd%2F&psig=AOvVaw33RLeTvZLhxQ4zw5XOFuvP&ust=1740082017218000&source=images&cd=vfe&opi=89978449&ved=0CBQQjRxqFwoTCOiOiObE0IsDFQAAAAAdAAAAABAE>

TDD

1 Beneficios de TDD

- 🐛 **Menos bugs:** Detecta errores antes de que lleguen a producción.
- 🧹 **Código más limpio y mantenible:** Refactoriza con confianza.
- 📖 **Documentación viva:** Los tests explican el comportamiento esperado.
- ⚙️ **Facilita el diseño:** Te obliga a pensar en la solución antes de codificar.
- ↺️ **Seguridad al hacer cambios:** Minimiza regresiones al modificar código.

2 Retos y Obstáculos Comunes

- 🕒 **Curva de aprendizaje:** Al principio puede parecer más lento.
- 🤖 **Mentalidad inversa:** Escribir tests *antes* del código requiere práctica.
- 🗑️ **Tests mal diseñados:** Pueden volverse difíciles de mantener.
- 💡 **Tentación de saltarse pasos:** Especialmente bajo presión de tiempo.
- 📦 **Cobertura vs. calidad:** Tener muchos tests no siempre significa tener buenos tests.

TDD

③ Buenas Prácticas para aprovechar TDD

Título:  Buenas Prácticas al usar TDD

Contenido (bullet points con tips prácticos):

-  **Escribe tests claros y concisos:** Si no se entienden, no ayudan.
-  **Testea comportamientos, no implementaciones:** Evita tests frágiles.
-  **Sigue el ciclo Red → Green → Refactor sin atajos.**
-  **Refactoriza tanto el código como los tests:** ¡Ellos también merecen limpieza!
-  **Empieza pequeño:** No intentes cubrir todo de golpe.
-  **Piensa en casos límite:** *"¿Qué pasa si me pasan datos vacíos o nulos?"*